

EntreVC Website & Portal — Product Requirements Document

Version: 2.0 (expanded from Club Brief v1.0) **Last Updated:** 21 August 2026 **Owner:** Entrepreneurship and Venture Capital Club, IIM Ahmedabad **Status:** Draft — ready for build

1. Product Overview

Product Name: EntreVC Web (public site) + EntreVC Console (admin portal)

Tagline: One permanent home for everything the entrepreneurship community at IIMA builds, learns and attends.

Objective: Replace the club's scattered posters, Google Forms and WhatsApp threads with a single public website backed by a self-serve admin console. A visitor should be able to understand the club, register for an event, browse student and alumni ventures, and download the playbook without messaging anyone. Every piece of content is published by the club team through the console, so the site survives annual handover without developer support.

The hard constraint that shapes every decision: the team turns over completely every year, and none of the incoming team is guaranteed to write code. If a feature can only be updated by editing a repo, it is the wrong feature.

2. Problem Statement

Core Problem

EntreVC's output — events, ventures, playbooks, mentor connections — currently lives in ephemeral channels. Event details reach students through WhatsApp forwards that expire from attention in a day. Registrations sit in one committee member's Google Form. Startups built by students and alumni have no shared directory, so investors and mentors have no credible view of the campus ecosystem. Past teams' knowledge assets vanish at handover.

Why Now?

Handover season is the natural moment: a site built now is populated by the incoming team through a full event cycle, so the content model gets stress-tested in year one rather than year three. The club also has an accumulating asset base (playbook, recordings, alumni venture list) that has no permanent home and is losing value the longer it stays in Drive folders.

Target Users

Segment	What they come for	Success looks like
Current students	Upcoming events, playbook, co-founder discovery, IIMA Ventures schemes	Registers for an event in under 60 seconds on mobile
Prospective students	Proof the ecosystem is real	Leaves with a concrete sense of scale (numbers, ventures, speakers)
Alumni founders	Visibility for their venture; staying connected	Their venture is listed accurately; they respond to a speaker invite

Investors / mentors / speakers	A credible, browsable view of campus ventures	Finds a relevant startup and a working contact route
Club team (internal)	Publishing during a busy term, often at 1 a.m. before an event	Publishes an event end-to-end from a phone in under 5 minutes

3. Solution & Features

High-Level Solution

A server-rendered Next.js site reading from Firestore, with a role-gated admin console at `/admin` in the same codebase. Public pages are statically generated and revalidated on publish, so the site is fast on 4G; the console writes to Firestore and triggers revalidation. Registrations, contact submissions and mailing-list signups are captured natively — no external form tools — and are exportable to CSV by the team.

Core Features (MVP Scope)

Feature 1: Events & Registration

Description: Upcoming and past event listings, a rich detail page per event, and a native registration flow with capacity limits, confirmation email and calendar file.

User Value: The single highest-frequency job on the site. A student sees a poster in a WhatsApp group, taps a link, and registers without leaving the page.

Sub-components:

- `/events` — upcoming events (chronological), past events (reverse-chronological), filter by event type
- `/events/[slug]` — hero image, title, date/time, venue, format (in-person / online / hybrid), speakers with photo + bio, agenda blocks, registration deadline, capacity indicator, register CTA
- Registration form: name, email (validate `@iima.ac.in` optionally per-event), phone, batch/organisation, role (student / alumni / external), plus up to 3 event-specific custom questions
- Past events convert to recap mode: recap text, photo gallery, embedded recording

Acceptance Criteria:

- Registration writes to Firestore and sends a confirmation email with an `.ics` attachment within 30 seconds
 - When `registeredCount >= capacity`, the form switches to a waitlist with a distinct confirmation message; capacity check is enforced server-side inside a transaction, not just in the UI
 - Registration closes automatically after `registrationDeadline` passes without any admin action
 - An event whose `endDateTime` has passed moves to the Past tab automatically at next revalidation
 - Duplicate registration (same email + event) is rejected with a friendly message, not a crash
 - Every event page emits a unique OG image and structured `Event` metadata
-

Feature 2: Startup Directory

Description: A filterable directory of ventures in two views — alumni ventures and current student ventures — plus a dedicated IIMA Ventures section covering schemes available to IIMA students.

User Value: For investors and mentors, this is the credibility artifact. For students, it's discovery — who's building what, and which institutional support they can tap.

Sub-components:

- `/startups` with a tab or toggle for Alumni / Student ventures
- Card: logo, name, one-line description, founders (name + batch), sector, stage, website, LinkedIn
- Filters: sector (multi-select), stage (multi-select), batch (range or multi-select), plus keyword search across name/description/founders
- `/startups/[slug]` detail page: longer description, founding story, team, funding stage, links
- `/iima-ventures` — overview, contact, highlight block, and a **schemes** list: each scheme with name, who it's for, what it offers, eligibility, application route, deadline (if any)

Acceptance Criteria:

- Filters combine (sector AND stage AND batch) and reflect in the URL query string so a filtered view is shareable
 - Filtering is client-side over a prefetched list — no loading spinner between filter clicks for directories under ~500 entries
 - All listings are created by the club team only; there is no public submission form in V1
 - A venture can be marked `featured: true` to surface on the homepage
 - No founder contact detail is published unless `contactConsent: true` is set in the admin record
-

Feature 3: Resources Library & Playbook

Description: The EntreVC Startup Playbook with a downloadable version, a tagged and searchable library of articles, books, podcasts and reports, an embedded YouTube feed, and optional email-gating on selected items.

User Value: This is what makes the site worth returning to between events, and what turns anonymous traffic into a mailing list.

Sub-components:

- `/resources/playbook` — chaptered web version with a download CTA (PDF)

- `/resources` — library grid with type filter (article / book / podcast / report / video / template) and tag filter, plus search
- Embedded YouTube channel feed (latest 6–9 videos)
- Gating: items with `gated: true` require an email before the download link resolves

Acceptance Criteria:

- Gated downloads issue a short-lived signed URL after email capture; the raw storage path is never exposed in HTML
 - Email captured through gating lands in the same `subscribers` collection as the mailing-list signup, tagged with source
 - Ungated resources are one click from list to source — no interstitial
 - YouTube feed degrades to a channel link if the API call fails; it never blocks page render
-

Feature 4: Team & Structure

Description: Current team with role, photo, batch and LinkedIn; a visual of the club structure and its verticals; an archive of past teams by year.

User Value: Externals need to know who to talk to; the archive gives outgoing teams a permanent record and makes handover feel like a lineage rather than a reset.

Acceptance Criteria:

- Team members group by vertical, ordered by a `displayOrder` field the admin controls
 - Switching to a past year (`/team/2025-26`) renders that year's roster from the same collection filtered by `academicYear`
 - Rolling over a year is an admin action (clone previous year as draft), not a data migration
-

Feature 5: Contact & Routing

Description: A contact page with club email, campus address, social links, role-specific contacts, and a categorised query form that auto-acknowledges the sender and routes to the right vertical.

User Value: Kills the "who do I email about sponsorship?" problem, which is currently answered by guessing.

Sub-components:

- Category selector: general / event / startup listing / sponsorship / speaking / mentorship
- Fields: name, email, batch or organisation, message
- Auto-acknowledgement to sender; routed notification to the vertical's mailbox based on a category → recipient map editable in the console

Acceptance Criteria:

- The category → recipient mapping is admin-editable; changing it requires no deploy
- Submissions persist to Firestore even if the outbound email fails, and failures surface in the console inbox with a retry action
- Honeypot field + rate limit by IP; no CAPTCHA in V1 unless spam appears

Feature 6: Admin Console

Description: Role-based login for the club team to create, edit, schedule and publish all content; view, filter and export registrations; read and resolve contact submissions; export the mailing list; manage media.

User Value: This is the feature that determines whether the site is alive in 2029. Everything else is downstream of it.

Roles:

Role	Can do
------	--------

admin	Everything, including user/role management and the category→recipient map
editor	Create, edit and publish all content types; export registrations
contributor	Create and edit drafts; cannot publish
viewer	Read-only access to registrations and submissions

Sub-components:

- Dashboard: next 3 events with live registration counts, unread submissions, recent activity
- Content editors for events, startups, resources, team, schemes, pages
- Draft → In Review → Published state machine with `scheduledPublishAt`
- Registrations table per event: search, filter, CSV export, mark attendance
- Submissions inbox: filter by category, mark resolved, assign
- Media library: upload, browse, copy URL, auto-compress on upload
- Users & roles screen (admin only)

Acceptance Criteria:

- Publishing triggers on-demand revalidation of the affected public routes within 10 seconds
 - `scheduledPublishAt` is honoured by a scheduled function running every 15 minutes
 - CSV export of registrations includes all custom question answers as columns
 - Handover is performed by changing role assignments on the users screen — no password sharing, no credential document
 - The console is usable on a 390px-wide phone for the three highest-frequency actions: publish an event, check registration count, read a submission
 - Every write records `updatedBy` and `updatedAt`; the last 20 changes per document are viewable
-

Feature 7: Homepage

Description: Positioning statement and club intro, next three events with direct register links, featured startups, latest resource, key numbers, mailing-list signup, entry points into every section, footer with social links.

Acceptance Criteria:

- The events strip pulls the next three chronologically and hides itself entirely if none are upcoming (never renders an empty shell)
 - Key numbers (events held, startups listed, alumni network size) are computed from live collection counts, with an admin override field per stat
 - Mailing-list signup requires only an email; double opt-in deferred to Phase 2
 - Above-the-fold content renders without client-side JS
-

Out of Scope (V1)

Deliberately excluded to keep the first ship tight. Each of these is cheap to add later and expensive to get wrong now.

- **Public startup self-submission** — the brief is explicit that the team curates listings; a submission queue is Phase 2
- **Mentor/investor 1:1 slot booking** — needs calendar integration and a no-show policy; Phase 2
- **EntreFair process board** — a distinct multi-stage workflow product; Phase 2, and worth its own spec
- **Blog & newsletter archive** — Phase 2, once the resources content model has proven itself
- **Site-wide global search** — Phase 2; per-section search covers V1 need at a fraction of the cost
- **Student login / member accounts** — no V1 use case justifies the auth surface for public users
- **Payments for paid events** — route to existing club payment rails; do not build a payment integration

- **Multi-language** — English only
 - **Native mobile apps** — responsive web only
-

4. User Flows

Primary Flow: Student registers for an event from a WhatsApp link

Scenario: A student taps an EntreVC link in a batch group on a 4G phone connection, ten minutes before a class.

Steps:

1. **User Action:** Taps the shared link `entrevc.in/events/vc-teardown-2026`
 - **System Response:** Statically generated event page renders; a rich OG preview already showed the poster and date in the chat
 - **UI Elements:** Hero image, date/venue block, sticky Register button on mobile
2. **User Action:** Scrolls through speakers and agenda, taps Register
 - **System Response:** Form expands inline (no route change, no modal that traps scroll)
 - **Conditional:** If `now > registrationDeadline` → button reads "Registrations closed", form is not rendered. If `registeredCount >= capacity` → button reads "Join waitlist" with a note about how waitlist promotion works.
3. **User Action:** Fills name, email, phone, batch; answers up to 3 custom questions; submits
 - **System Response:** Server action validates, runs a Firestore transaction that re-checks capacity and

increments `registeredCount`, writes the registration, and queues the confirmation email

- **Conditional:** Duplicate email for the same event → "You're already registered — check your inbox" with a resend-confirmation link

4. **System:** Sends confirmation email with `.ics` attachment and an add-to-calendar link

- **UI Elements:** Inline success state with the event date, venue, and a second add-to-calendar button for users who ignore email

Success State: The student is registered, has the event in their calendar, and never left the page or opened a Google Form.

Flow: Club member publishes an event at 1 a.m. from a phone

1. **User Action:** Opens `/admin`, signs in with Google using their institute account

- **System Response:** Firebase Auth checks a custom claim for role; unrecognised accounts get a "request access" screen, not a raw error

2. **User Action:** Taps New Event, fills title, date, venue, format, capacity, deadline; uploads a poster

- **System Response:** Poster is compressed and converted to WebP on upload; a slug is auto-generated from the title and shown as editable

3. **User Action:** Adds speakers and agenda rows; taps Save Draft

- **System Response:** Draft saved, visible only in console and at a preview URL with a signed token

4. **User Action:** Taps Publish (or sets `scheduledPublishAt`)

- **System Response:** Status flips to published, on-demand revalidation fires for `/`, `/events`, `/events/[slug]`; a shareable link with OG preview is shown with a copy button
- **Conditional:** A `contributor` sees Submit for Review instead of Publish; an `editor` or `admin` gets a review queue notification

Success State: Event is live, indexed and shareable in under 5 minutes, from a phone, with no developer involved.

Flow: Investor browses ventures and makes contact

1. Lands on `/startups` from a LinkedIn share
2. Filters to sector = Fintech, stage = Seed → URL updates to `/startups?sector=fintech&stage=seed`, list filters instantly
3. Opens a venture detail page, reads the story, clicks through to the venture's own site
4. Returns, goes to `/contact`, selects category = mentorship, submits a message
5. Auto-acknowledgement arrives; the routed notification lands in the mentorship vertical's mailbox and appears in the console inbox

Success State: A qualified external contact reaches the right club member without knowing any names in advance.

5. Tech Stack

Frontend

Choice: Next.js 15 (App Router) + React 19 + TypeScript + Tailwind

CSS 4 Rationale: The brief specifies a dynamic framework; App Router gives static generation with on-demand revalidation, which is exactly the shape of this problem (content changes rarely, reads are frequent, publishes must appear immediately). Server Components keep the JS

bundle small enough to hit the 4G LCP target. **Key capabilities used:** Static generation with ISR, `revalidatePath` from Server Actions, `generateMetadata` for per-page OG tags, `next/image` for automatic image optimisation, `opengraph-image.tsx` for dynamic preview cards.

Backend

Choice: Next.js Server Actions and Route Handlers + Firebase Cloud Functions for scheduled and async work **Rationale:** Registration, contact and signup writes are simple mutations that belong next to the UI. Only two jobs genuinely need to run outside a request: scheduled publishing and email retries — those go to Cloud Functions. **Hosting:** Vercel for the app; Cloud Functions in `asia-south1` for scheduled jobs.

Database

Choice: Firebase Firestore (as specified in the club brief) + Firebase Storage for media **Rationale:** Document model fits content with variable shapes (agenda blocks, custom questions). Security rules give a clean read-public / write-role-gated split. Free tier comfortably covers club-scale traffic.

Schema Overview:

```
events/{eventId}
  slug, title, subtitle, description (rich text)
  startDateTime, endDateTime, timezone
  venue { name, address, mapLink }, format: 'in_person'|'
  eventType, heroImage, gallery[], recordingUrl, recapTex
  speakers[] { name, title, org, photo, bio, linkedin }
  agenda[] { time, title, description, speakerRefs[] }
  capacity, registeredCount, waitlistCount, registrationD
  registrationFields[] { id, label, type, required, optio
  restrictToInstituteEmail: boolean
  status: 'draft' | 'in_review' | 'published', scheduledPubli
  featured, createdBy, updatedBy, updatedAt
```

```
registrations/{registrationId}
  eventId, eventSlug, name, email (lowercased), phone
  batch, organisation, attendeeType: 'student'|'alumni'|
  customAnswers { fieldId: value }
  status: 'confirmed'|'waitlisted'|'cancelled'
  consentGiven: boolean, attended: boolean
  createdAt, confirmationEmailStatus: 'sent'|'failed'|'pe

startups/{startupId}
  slug, name, logo, tagline, description
  type: 'alumni'|'student'
  founders[] { name, batch, linkedin, photo, contactConse
  sector[], stage, foundedYear, website, linkedin
  featured, status, updatedBy, updatedAt

ventureSchemes/{schemeId} // IIMA Ventures schem
  name, summary, whoItIsFor, whatYouGet, eligibility
  applicationLink, deadline, displayOrder, status

resources/{resourceId}
  slug, title, type: 'article'|'book'|'podcast'|'report'|
  description, url, storagePath, coverImage
  tags[], gated: boolean, featured, publishedAt, status

playbookChapters/{chapterId}
  order, title, body (rich text), downloadPath

teamMembers/{memberId}
  name, role, vertical, batch, photo, linkedin
  academicYear: '2026-27', displayOrder, status

submissions/{submissionId}
  category, name, email, batchOrOrg, message
  routedTo, status: 'new'|'in_progress'|'resolved'
  assignedTo, createdAt, emailStatus

subscribers/{subscriberId}
  email (lowercased, unique), source: 'homepage'|'gated_r
  consentGiven, consentTimestamp, active, createdAt
```

```
siteSettings/{singleton}
  positioningStatement, keyNumbers { eventsHeld, startups
  socialLinks, contactEmails, categoryRoutingMap, roleCon

users/{uid}
  email, name, role, vertical, active, lastLogin
```

Critical indexes: `events(status, startDateTime)`,
`startups(type, status, sector)`, `registrations(eventId,`
`createdAt)`, `resources(status, type, publishedAt)`,
`teamMembers(academicYear, displayOrder)`.

Authentication

Choice: Firebase Auth with Google provider, restricted to approved accounts, roles via custom claims **Rationale:** Institute Google accounts already exist; custom claims let security rules and middleware read role without an extra fetch. Only the console needs auth — public pages have zero auth surface.

Deployment & Infrastructure

Platform: Vercel (app), Firebase (data, storage, functions) **CI/CD:** Auto-deploy from `main`; every PR gets a preview URL **Environments:** Two Firebase projects — `entrevc-prod` and `entrevc-dev` — so a new team can break things safely **Domain:** Club domain on Vercel with automatic HTTPS

Key Dependencies

- `next (15.x)` — framework
- `firebase + firebase-admin (12.x / 13.x)` — client and server SDKs
- `react-hook-form + zod` — form state and shared client/server validation

- `@tanstack/react-table` (8.x) — registrations and submissions tables with sort/filter/export
- `resend` — transactional email (confirmations, acknowledgements, routing); SendGrid is an equivalent substitute
- `react-email` — email templates as components, so the team can preview them
- `ics` (3.x) — calendar file generation
- `@tiptap/react` — rich text editing in the console
- `browser-image-compression` — client-side compression before upload
- `papaparse` — CSV export
- `tailwindcss` (4.x) + `lucide-react` — styling and icons
- `date-fns` (4.x) — date handling, all stored as UTC, displayed in IST

Suggested Structure

```

app/
  (public) /
    page.tsx                                # Home
    events/page.tsx
    events/[slug]/page.tsx
    events/[slug]/opengraph-image.tsx
    startups/page.tsx
    startups/[slug]/page.tsx
    iima-ventures/page.tsx
    resources/page.tsx
    resources/playbook/page.tsx
    team/page.tsx
    team/[year]/page.tsx
    contact/page.tsx
  admin/
    layout.tsx                             # role gate
    page.tsx                               # dashboard
    events/, startups/, resources/, team/, schemes/
    registrations/[eventId]/page.tsx
    inbox/page.tsx
    media/page.tsx

```

```
    subscribers/page.tsx
    users/page.tsx                # admin only
  api/
    revalidate/route.ts
    ics/[eventId]/route.ts
    resources/[id]/download/route.ts
  lib/
    firebase/{client,admin}.ts
    actions/{registration,contact,subscribe,content}.ts
    schemas/                      # zod schemas shared by c
    email/                        # react-email templates
  components/{ui,public,admin}/
  functions/                      # scheduled publish, email
  firestore.rules
```

6. Success Metrics

Primary Metric: Event registrations completed on-site

Definition: Count of registrations with status = confirmed created through the site, per term **Target:** 80% of all EntreVC event registrations flow through the site rather than Google Forms by the end of the first full term **Why This Matters:** It is the only metric that proves the site replaced the old workflow rather than sitting alongside it. If forms persist, the site has failed at its actual job.

Secondary Metrics

1. Admin self-sufficiency

- Definition: Percentage of content published in a month with zero developer involvement
- Target: 100% after the first month
- Measurement: Count of published documents vs. count of

code commits touching content

2. Registration funnel conversion

- Definition: Event page views → registration submissions
- Target: > 25% on events promoted via WhatsApp
- Measurement: GA4 funnel using a `registration_complete` event

3. Mailing list growth

- Definition: Net new active subscribers per month
- Target: 150 in term one
- Measurement: Count on `subscribers where active = true`, by month

4. Directory engagement from external traffic

- Definition: Sessions from LinkedIn/direct that view ≥ 2 startup pages
- Target: 200 per term
- Measurement: GA4 segment on referrer + page path

5. Performance in the field

- Definition: Real-user LCP at p75
- Target: < 2.5s
- Measurement: Vercel Speed Insights

Measurement Approach

Tools: Vercel Analytics + Speed Insights, Google Analytics 4, Firestore
aggregate queries surfaced on the admin dashboard **Implementation:**
Fire GA4 events for `register_start`, `registration_complete`,
`resource_download`, `newsletter_signup`, `startup_view`,
`contact_submit` **Review cadence:** The team reviews the admin
dashboard weekly; a fuller metric review at each term boundary and at
handover.

7. Technical Considerations

Performance

- **LCP:** < 2.5s on 4G (brief requirement) — met via static generation, WebP/AVIF via `next/image`, and self-hosted fonts with `next/font`
- **Server action response:** < 500ms for registration writes; email dispatch is queued, never blocking the response
- **Bundle:** < 150KB JS on the homepage; the console can be heavier since it is authenticated and used on wifi
- **Concurrency:** Must survive a spike of ~500 concurrent registrations in the ten minutes after a WhatsApp blast — the capacity transaction is the one contended write, so it must be a Firestore transaction, not a read-then-write

Browser & Device Support

- Chrome, Safari, Firefox, Edge — last two versions
- Responsive from 360px upward (brief requirement); mobile-first, since the dominant entry point is a phone
- Admin console fully usable at 390px for publish, registration-count and inbox actions

Accessibility

- **Target:** WCAG 2.1 Level AA
- Keyboard navigation throughout, visible focus rings, 4.5:1 text contrast, semantic headings, alt text required on every image upload in the console (enforced as a required field), form labels tied to inputs, errors announced to screen readers

Security & Privacy

- **Auth:** Firebase Auth (Google), roles as custom claims, `/admin/*`

gated in middleware *and* at the Firestore rules layer — never in the UI alone

- **Firestore rules:** public read only on documents where `status == 'published'`; all writes require an authenticated role claim; `registrations`, `submissions` and `subscribers` are write-only for the public and read-only for authorised roles
- **Consent:** An explicit consent checkbox on registration and signup, with the consent text and timestamp stored on the record
- **Publication consent:** No personal contact detail appears publicly unless the corresponding record carries written approval — enforced by a `contactConsent` flag that gates rendering
- **Abuse:** Honeypot fields, IP rate limiting on all public writes, file type and size limits on uploads
- **Secrets:** Server-only Firebase Admin credentials and email API keys as Vercel environment variables; never in `NEXT_PUBLIC_*`
- **Retention:** Registration data retained for two academic years, then exported and purged — a documented annual task in the handover checklist

Scalability

Club-scale traffic (low thousands of monthly visitors) sits well inside Firebase's free tier. The two things that would break first are unbounded directory reads and media storage growth: mitigate by paginating any collection past 500 documents and compressing images on upload. Aggregate counters (`registeredCount`) are maintained incrementally rather than counted on read.

8. Assumptions & Risks

Assumptions

1. **The club team will maintain content.** The entire architecture assumes an active editor. If nobody publishes, the site decays into

the poster wall it replaced.

2. **The team holds institute Google accounts** usable for Firebase Auth.
3. **Startup listings stay curated.** Volume stays in the hundreds, so client-side filtering is viable and no submission queue is needed in V1.
4. **Events are free**, or paid through existing club rails; no payment integration is required.
5. **Someone owns the domain and DNS**, and that ownership transfers at handover.
6. **The playbook exists as content** ready to be entered — the site does not create it.

Technical Risks

Risk	Impact	Likelihood	Mitigation
Registration race at capacity boundary oversells the venue	High	Medium	Enforce capacity inside a Firestore transaction; treat any client-side check as advisory only
Confirmation emails land in spam or Promotions	High	High	Authenticate the sending domain (SPF, DKIM, DMARC); always show the confirmation inline as well; include a resend link
Firestore free-tier read limits hit during an event-day traffic spike	Medium	Low	Static generation means most visitors trigger zero reads; set a billing alert
Rich text editor produces inconsistent or unsafe markup	Medium	Medium	Constrain the Tiptap toolbar to a fixed set of nodes; sanitise on render

Vendor lock-in across two platforms (Vercel + Firebase)	Medium	Low	Keep data access behind <code>lib/firebase/</code> * so the data layer is swappable; schedule regular Firestore exports to Cloud Storage

Adoption & Continuity Risks

Risk	Impact	Likelihood	Mitigation
Incoming team can't or won't use the console — the single biggest threat to the project	High	Medium	Ruthless console simplicity; a 2-page written handover guide plus a screen recording; a live handover session each year; the console itself explains states rather than assuming knowledge
Team keeps using Google Forms out of habit	High	Medium	Make the site path strictly faster than creating a form; export parity (CSV with all fields) so nothing is lost by switching
Content goes stale — outdated "upcoming" events on the homepage	Medium	High	Automatic past/upcoming transitions with no admin action; a dashboard warning when nothing is scheduled in the next 30 days
Alumni ventures listed without permission	High	Low	Require an approval flag on every listing before publish; a documented takedown route on the contact page

Domain or hosting account lost at handover	High	Medium	Register the domain and both platform accounts to a club-owned email, never a personal one; record this in the handover checklist
--	------	--------	---

9. Implementation Roadmap

Phase 1: Core MVP — Target 10–12 working days

The order matters: content model first, then the console, then the public pages. Building public pages first against a guessed schema is the classic way to lose three days to rework.

Sprint 1 — Foundation (Days 1–3)

- ☐ Next.js 15 + TypeScript + Tailwind scaffold, design tokens, base layout — 3h
- ☐ Firebase projects (dev + prod), Firestore collections, indexes, security rules — 4h
- ☐ Firebase Auth with Google, custom claims, `/admin` middleware gate — 4h
- ☐ Zod schemas for every entity, shared client/server — 3h
- ☐ Media upload to Firebase Storage with client-side compression — 3h
- ☐ Shared UI kit: form fields, cards, tables, status badges, empty states — 5h

Sprint 2 — Console (Days 4–6)

- ☐ Event editor: full CRUD, speakers, agenda, custom fields, draft/publish — 8h
- ☐ Startup and scheme editors — 5h
- ☐ Resource and playbook-chapter editors, gating toggle — 4h

- ☐ Team editor with year rollover — 3h
- ☐ Dashboard: upcoming events, registration counts, unread submissions — 3h
- ☐ Users & roles screen — 3h

Sprint 3 — Public site (Days 7–9)

- ☐ Homepage with all seven blocks — 5h
- ☐ Events list + detail + past/recap mode — 6h
- ☐ Registration flow: form, transaction, waitlist, duplicate handling — 6h
- ☐ Confirmation email + `.ics` generation — 4h
- ☐ Startup directory with filters and detail pages — 6h
- ☐ IIMA Ventures page with schemes — 2h
- ☐ Resources library, playbook, YouTube embed, gated download — 5h
- ☐ Team page with archive; contact page with routed form — 5h

Sprint 4 — Operations & launch (Days 10–12)

- ☐ Registrations table with filters and CSV export — 4h
- ☐ Submissions inbox with routing map and status — 3h
- ☐ Subscribers list and export — 2h
- ☐ Per-page metadata, dynamic OG images, sitemap, robots — 4h
- ☐ GA4 + Vercel Analytics, event instrumentation — 2h
- ☐ Accessibility pass and Lighthouse tuning to LCP target — 4h
- ☐ Content seeding with the team (this is a team activity, not a dev task) — 4h
- ☐ Domain, DNS, production deploy, handover guide + screen recording — 4h

Definition of Done: A club member with no code access can publish an event from a phone; a student can register and receive a confirmation with a calendar file; the team can export registrations to CSV; the startup directory filters and is shareable; the site scores LCP < 2.5s on a throttled 4G Lighthouse run; a written handover guide exists.

Phase 2: Enhancements (Post-launch, term 1)

Ordered by the club brief's own "additional recommendations", with the cheapest high-value items first.

- Blog and newsletter archive (extends the existing content model — smallest lift)
- Site-wide global search across events, startups, resources and blog
- Public startup submission with an admin approval queue
- Mentor and investor connect: 1:1 slot booking with calendar sync and no-show handling
- Newsletter sending from the console, or a clean Mailchimp/Buttontdown sync
- Attendance check-in via QR at the venue
- Double opt-in on the mailing list

Phase 3: Scale (Future)

- **EntreFair board**: full process management — applications, shortlisting, stall allocation, judging, results. This deserves its own PRD; it is a workflow product, not a page.
 - Co-founder matching for students
 - Alumni-founder self-service profile claiming and editing
 - Public API or embeddable widgets for the startup directory
 - Analytics dashboard for the team inside the console
-

10. Open Questions & Decisions Needed

- ☐ **Domain**: What is the final domain, and which club-owned account will hold registration and DNS?
- ☐ **Email sender**: Which address sends transactional mail, and who controls the DNS records needed to authenticate it? This blocks deliverability work and should be resolved before Sprint 4.
- ☐ **IIMA Ventures relationship**: Is the schemes content supplied and approved by IIMA Ventures, and who signs off on updates? Needs a named owner.
- ☐ **Alumni venture consent**: Blanket approval process, or per-

listing written consent? The privacy requirement in the brief implies the latter — confirm.

- ☐ **Institute-email restriction:** Should registration be limited to `@iima.ac.in` by default, per-event, or never? The schema supports per-event; the default needs a decision.
 - ☐ **Event capacity behaviour:** Should waitlisted registrants be auto-promoted on cancellation, or promoted manually by the team?
 - ☐ **Playbook format:** Web-first with a generated PDF, or an uploaded PDF with a web summary? Affects Sprint 2 scope.
 - ☐ **Who builds it:** Current-team members with Claude Code, or a contracted build? This changes nothing about the spec but everything about the handover plan.
-

Appendix

Handover Checklist (annual, ~30 minutes)

1. Outgoing admin adds incoming team in Console → Users, assigns roles
2. Incoming lead is promoted to `admin`; outgoing lead is set to `viewer`, then deactivated after two weeks
3. Verify club-owned email still controls Vercel, Firebase, domain registrar and the email provider
4. Roll over the team page: clone previous year as draft, edit, publish
5. Export and archive last year's registrations; purge records older than two academic years
6. Walk through the handover guide and screen recording together
7. Update `siteSettings` → contact emails and role contacts

Glossary

- **P0:** Required for launch; the site does not ship without it
- **ISR / on-demand revalidation:** Rebuilding a specific static page

when its content changes, rather than on a timer

- **Custom claim:** A role stored on the Firebase Auth token, readable by security rules without a database fetch
- **Gated resource:** A download that requires an email address before the file link resolves
- **Vertical:** A functional sub-team within the club (e.g. events, ventures, outreach)